RTL for AXI4 (Master & Slave) to PCIe 6 Link

AXI4-Stream to PCIe 6.0 Bridge — Stage 1 Design Report

Aditya — Swaray — Kshitij

SARCathon 2026 – SemiCon Hackathon

Abstract

This report describes the RTL design, module partitioning, inter-block interfaces, and verification results for an AXI4-Stream-to-PCIe 6.0 bridge targeting two device-to-host DMA use cases: SmartNIC network-packet ingestion and ML-inference-accelerator output streaming. The design is split across three cooperating layers – an *interconnect layer* (AXI4-Stream ingestion, arbitration, clock-domain crossing), a *link layer* (session/address management, FLIT framing, reliability), and a *protocol-to-PHY layer* (lane encoding, PCIe reset/config/MSI-X sideband) – each independently simulated and joined by a cross-layer system-level integration (`Final/system_top.v`). This document covers what has been built and verified to date (Stage 1: RTL + testbench + simulation results) and states plainly what remains open before the design can be considered complete or before Stage 2 (synthesis, physical implementation, GDSII) can begin.

Contents

1 Architecture Understanding **3**
1.1 Problem statement . . . 3
1.2 End-to-end pipeline . . . 3
1.3 Clock domains . . . 3

2 Implemented Blocks **4**
2.1 Interconnect layer . . . 4
2.2 Link layer . . . 5
2.3 Protocol–PHY layer . . . 6

3 Interfaces **7**
3.1 Interconnect layer → link layer (`interconnect_top` output) . . . 7
3.2 Session-arm handshake (host → interconnect layer, per channel) . . . 7
3.3 Link layer ↔ PHY layer (`data_link_layer_top` ↔ `pcie_phy_link_datapath`) . . . 7
3.4 PCIe control-plane sideband (`pcie_link_top`) . . . 8

4 System-Level Integration (`Final/system_top.v`) **8**
4.1 Data path . . . 8
4.2 Synthesizable auto-SOF FSM . . . 8
4.3 Control-plane wiring . . . 8

5 Verification Scenarios and Key Results **8**
5.1 Interconnect layer – unit-level . . . 8
5.2 Cross-team integration . . . 9
5.3 Protocol–PHY layer . . . 10
5.4 Link layer . . . 10

6 Conclusion **10**

1 Architecture Understanding

1.1 Problem statement

An AXI4-to-PCIe bridge is required to move data from FPGA-fabric-side AXI4-Stream producers directly into host server memory over a PCIe 6.0 link, for two workloads:

- **SmartNIC / network packet processing** – incoming Ethernet packets, streamed continuously into a host buffer.
- **ML inference accelerator** – inference output (weights or pixel data) streamed into a host buffer.

Both use cases are strictly *device-to-host, one-directional* writes: the FPGA never reads host memory in this design. This observation shaped the entire interconnect-layer scope (Section 2.1) – no AXI4 memory-mapped read/write slave port, no AXI4 master DMA-read engine, only two independent AXI4-Stream ingest ports.

1.2 End-to-end pipeline

The design is organized as three pipeline stages, each owned by a different engineer, connected by narrow, explicitly documented hand-off contracts:

Stage	Function	Owner
Interconnect layer	AXI4-Stream ingest ×2, weighted round-robin arbitration, CDC into the PCIe clock domain	Aditya
Link layer	Per-session base-address gating, write-data buffering, replay buffer, FLIT framing, CRC/FEC status	Swaray
Protocol–PHY layer	Gray coding, PAM4 mapping, 4-lane striping/SOF framing, PCIe reset controller, config space, MSI-X	Kshitij

A single host physical address is never computed inside the interconnect layer. Instead, the interconnect layer produces only a small per-channel *running byte offset*; the real 64-bit host address (`base_addr_ch[N]` + `session_offset`) is combined downstream, once a channel's base address has been armed by the host via the session-arm handshake described in Section 2.1. This keeps the interconnect layer's datapath narrow (an `OFFSET_WIDTH`-bit field rather than 64 bits) and avoids storing the same address in two places.

1.3 Clock domains

Two clock domains are in play throughout: `aclk` (AXI-side ingest, target ≤ 250 MHz) and `pcie_clk` (PCIe link-side). The *only* sanctioned crossing point between them is a gray-code asynchronous FIFO inside the interconnect layer (`cdc_fifo_async.v`); everything downstream of that FIFO, including the link layer and PHY layer built so far, lives in the `pcie_clk` domain. One additional, currently-unresolved CDC surface is discussed in Section **??**.

2 Implemented Blocks

2.1 Interconnect layer

Module	Function
`axi4s_ingest_port.v`	One instance per channel. AXI4-Stream slave handshake, byte-accurate session-offset tracking via a synthesizable `popcount` of `TKEEP`, look-ahead overflow guard, and a decoupled `req`/`accepted` pair that distinguishes "granted" from "handshake actually completed" (the fix for a duplicate-write bug found during integration).
`rr_arbiter_2ch.v`	Weighted round-robin arbiter across the two channels, credit-based with independent per-channel refill (fixes a joint-refill deadlock present in an earlier version), plus a starvation guard that forces a grant after `STARVE_CYCLES`. Grant outputs are registered, not combinational, for timing closure into the crossbar mux.
`cdc_fifo_async.v`	Generic gray-code asynchronous FIFO, the sole clock-domain crossing point. First-word-fall-through read semantics (fixes a registered-read/FWFT timing mismatch found during development) and an unconditional look-ahead pointer for the `full` flag (fixes an earlier combinational-loop bug that hung simulation).
`interconnect_top.v`	Wires the above into the complete layer: 2 ingest ports → crossbar mux → arbiter-driven select → CDC FIFO → output port.
`session_arm_ctrl.v`	Per-channel 3-state FSM (`IDLE`→`LATCH`→`ACTIVE`) implementing the session-armed handshake: host writes `base_addr`/`tc` then a write-1-to-set `session_arm` bit; hardware latches the address into shadow registers and asserts `session_valid` as a level signal for the duration of the session.
`bridge_top.v`	First cross-team integration point: wires `interconnect_top` → `write_data_buffer` (link layer), each channel gated by its own `session_arm_ctrl` instance.

2.2 Link layer

Module	Function
write_data_buffer.v	AW+W-equivalent merge buffer sitting directly downstream of the interconnect layer's CDC output; backpressure via a configurable high-watermark; the module this design report's integration point (bridge_top) terminates at.
replay_buffer.v	Stores transmitted FLITs and replays on NAK; passes TUSER (stream_id/session_offset) through combinationally rather than storing it, since on retransmit the same information is reconstructible from the FLIT sequence number.
link_layer_tx.v / link_layer_rx.v	Per-channel FLIT assembly and CRC/FEC framing, with per-channel-ready propagation (s_axis_tready_ch) since the interconnect layer's arbiter interleaves both channels' beats onto one bus – the FLIT-assembly logic must maintain fully independent per-channel state.
DLLP_parser.v	Parses ACK/NAK DLLPs from the RX stream and feeds sequence numbers back to the TX-side replay buffer.
data_link_layer_top.v	Wraps TX/RX pipelines, replay buffer, and DLLP parser; exposes a plain 256-bit m_phy_tx_* / s_phy_rx_* system-bus interface toward the PHY layer, plus crc_err/fec_err status.
Also present, not yet wired into the system top: tag_manager.v, credit_manager.v, tlp_packetizer.v / tlp32_to_axis_packetizer.v, address_translation.v, ar_request_splitter.v, axi_pcie_reorder_buffer.v, tx_engine.v, pcie_rx_engine.v, fec_encoder_decoder.v, flit_framer.v	Individually built and unit-tested (see Section 5); these implement the TLP-header, tag-management, credit-flow-control, and read-path machinery that a full PCIe transaction layer needs but that is not yet part of the system-level data path (see Section **??**).

2.3 Protocol–PHY layer

Module	Function
`gray_enc/gray_dec`	2-bit Gray encode/decode, exact inverses (formally checked – Section 5).
`pam4_map/pam4_demap`	Maps a Gray-coded 2-bit symbol to one of four normalized voltage levels $\{-3, -1, +1, +3\}$ and back.
`byte_to_symbol/ symbol_to_byte`	Serializes a byte into four 2-bit PAM4 symbols on TX and reassembles on RX.
`flit_lane_striper/ flit_lane_destriper`	Distributes a 256-bit FLIT across 4 lanes in byte-interleaved round-robin order over 8 cycles, and the inverse reassembly.
`tx_elastic_deskew`	Per-lane elastic buffering (4× `elastic_fifo`) with a shared priming threshold, needed because the striper emits one byte/lane/cycle while the lane pipes need 4 cycles per byte to serialize.
`flit_sof_strip`	FLIT-mode SOF token (`0xFB`) detection and 8-cycle payload pass-through, with a stated (not solved) limitation around `0xFB` appearing as genuine payload.
`ts_ordered_set_ detector/lane_ reversal_detector/ lane_reversal_latch`	TS1/TS2 ordered-set detection per lane, majority-based lane-reversal decision, and a latch that locks the decision once on first confirmed training (real PCIe re-evaluates only on retraining).
`pcie_reset_ controller`	PERST#, hot-reset, and FLR handling; `datapath_rst_n` is deliberately *never* gated on `link_trained` (would deadlock, since the training logic itself lives in that reset domain); `axi_rst_n` deasserts only after link training *and* a first UpdateFC DLLP.
`pcie_config_space / msix_controller`	Type 0 config-space header, up to 6 BAR registers, Device Control/Link Status registers; MSI-X vector table (up to 2048 entries) with per-vector masking and PBA.
`pcie_phy_link_ datapath.v`	Synthesizable top wrapping the striper/deskew/lane-pipe/ swap/SOF-strip/destriper chain behind a plain FLIT-in/FLIT-out interface, with a synthesizable auto-SOF FSM replacing what was previously only testbench stimulus. See Section 4.

3 Interfaces

3.1 Interconnect layer → link layer (`interconnect_top` output)

Signal	Width	Notes
`m_tdata`	512	AXI4-Stream payload
`m_tkeep`	64	Byte-valid mask
`m_tuser`	`OFFSET_WIDTH` +2 (26 at production width 24)	{`tlast_flag`, `session_offset`, `stream_id`} – packed as a single bus (originally two separate signals; repacked to match the link layer's single-`tuser`-bus requirement)
`m_tlast`	1	End of session
`m_tvalid/m_tready`	1/1	First-word-fall-through handshake

Note: 1) `m_tlast` and the `tlast_flag` bit packed into `m_tuser` carry the same value each cycle – `interconnect_top.v` mirrors `m_tlast` into `tuser`'s MSB (`assign m_tuser = {m_tlast, m_session_offset_i, m_stream_id_i};`), so it is one signal exposed twice, not two independent ones.

Note: 2) The interconnect layer arbitrates *per beat*, not per session – with default 50/50 weights, SmartNIC and ML beats interleave almost every cycle (e.g. C0, C1, C0, C1, ...) rather than one channel completing before the other starts. Downstream FLIT-assembly logic must therefore keep fully independent per-channel state, which is why `link_layer_tx.v` carries per-channel ready (`s_axis_tready_ch`).[4pt]

3.2 Session-arm handshake (host → interconnect layer, per channel)

Signal	Width	Notes
`base_addr_in`	64	Host-supplied destination base address
`tc_in`	3	Traffic class, latched per-session (not per-beat)
`session_arm`	1	Write-1-to-set pulse; hardware clears it internally
`session_valid`	1	**Level**, not a pulse – held high for the session duration (a design decision reached after review; the ingest port gates directly on this level)

3.3 Link layer ↔ PHY layer (`data_link_layer_top` ↔ `pcie_phy_link_datapath`)

Signal	Width	Notes
`m_phy_tx_valid/data/sof/eof`	1/256/1/1	One 256-bit word = one FLIT; `sof` and `eof` assert together with the single transfer cycle. No ready/back-pressure signal exists on this interface today (see Section 4).
`s_phy_rx_valid/data/sof/eof`	1/256/1/1	Same convention, receive direction.
`crc_err/fec_err`	1/1	Status outputs, surfaced to the system top.

3.4 PCIe control-plane sideband (pcie_link_top)

Reset requests (hot_reset_request, flr_request, updatefc_received), TS1/TS2 training byte stimulus (4 lanes), CfgRd/CfgWr (address/data), and MSI-X table read/write plus a DMA-complete pulse/vector pair that triggers an MSI-X MWr. All exposed as plain top-level ports on system_top.v – see Section 4.

4 System-Level Integration (Final/system_top.v)

4.1 Data path

system_top.v instantiates, in order: bridge_top (interconnect layer + write_data_buffer) → data_link_layer_top (replay buffer, TX/RX FLIT framing, CRC/FEC) → pcie_phy_link_datapath (Gray/PAM4 lane encode-decode, striping/de-striping, SOF framing). Because no real external SerDes exists anywhere in this repository (consistent with pcie_phy_datapath.v's own *"ideal channel: no noise/attenuation modeled yet"* comment), the reconstructed FLIT output of pcie_phy_link_datapath is looped directly back into data_link_layer_top's own PHY RX input – exercising the complete encode/decode chain in simulation without a physical wire.

4.2 Synthesizable auto-SOF FSM

SOF-token injection ahead of each FLIT (Section 5, tb_end_to_end_datapath) is done by a 4-state FSM (IDLE→SOF →PUSH→DRAIN) that:

1. captures each incoming FLIT into a small input FIFO (since m_phy_tx_* has no ready signal to push back on the link layer – a stated, not solved, gap; overflow is made visible via tx_fifo_overflow rather than silently dropped);
2. asserts exactly one SOF cycle ahead of the real payload;
3. pulses the striper's valid_in for one cycle; and
4. holds until the destriper on the far side confirms completion before accepting the next FLIT (the chain is not built to pipeline multiple FLITs concurrently).

4.3 Control-plane wiring

system_top additionally instantiates pcie_link_top (reset controller, TS1/TS2 training, config space, MSI-X). datapath_rst_n drives both data_link_layer_top's and pcie_phy_link_datapath's reset (rather than the free-running pcie_rstn either module would otherwise use), and lane_reversed_locked drives the PHY datapath's lane_reversed_in (rather than a tied-0 placeholder). Because pcie_reset_controller deliberately never gates datapath_rst_n on link_trained (its own header comment states this is to avoid a deadlock, since the training logic lives inside that same reset domain), the data path still runs correctly even though nothing in this repository currently drives real TS1/TS2 byte stimulus into the training ports – only axi_rst_n, which is not consumed anywhere in this data path yet, would remain gated indefinitely.

5 Verification Scenarios and Key Results

5.1 Interconnect layer – unit-level

All testbenches run with Icarus Verilog; all pass.

Testbench	What it checks	Result
`tb_axi4s_ingest_port.v`	Session-offset tracking, TLAST ending a session, overflow guard aborting *before* exceeding `session_limit`, back-pressure holding TREADY low	PASS
`tb_rr_arbiter_2ch.v`	50/50 default split, 3:1 weighted ratio, starvation guard for a weight-0 channel, single-requester never stalling, grant0/grant1 mutual exclusion	PASS
`tb_cdc_fifo_async.v`	Data integrity across two genuinely asynchronous clocks (7 ns/13 ns periods, deliberately unrelated periods), in-order delivery, no loss	PASS
`tb_interconnect_top.v`	Full-stack, two channels driven simultaneously across independent `aclk`/`pcie_clk`, correct `stream_id` tagging, correct per-channel `session_offset` sequencing	PASS

Key measured result (interconnect master log): a 128-beat, two-channel interleaved run completed with `wr_cnt=128, rd_cnt=128, accepted0_cnt=64, accepted1_cnt=64, recv0=64, recv1=64, errors=0` – zero drops, confirmed via three independent internal counters reconciling exactly.

5.2 Cross-team integration

`tb_bridge_smoke.v` drives two beats (one mid-session, one TLAST) into channel 0 of `bridge_top` and checks both the raw data and the packed `TUSER` field (`stream_id`, `session_offset`, `tlast_flag` mirror) exiting `write_data_buffer` match expectations bit-for-bit – **PASS**.

`tb_system_top_smoke.v` extends this one stage further: a single beat is pushed into channel 0, and the test waits (with a 2000-cycle timeout, sized against the PHY chain's ∼100+ cycle-per-FLIT latency) for a reconstructed FLIT to emerge from `pcie_phy_link_datapath`'s loopback, with `sof`/`eof` asserted and `crc_err`/`fec_err`/ `phy_tx_fifo_overflow` all clear. This is stated explicitly as a *connectivity* smoke test, not a data-correctness test against the original AXI4-Stream payload – no TLP packetizer sits between `bridge_top` and `data_link_layer_top` yet (see Section **??**), so the reconstructed FLIT does not carry a real TLP header.

Result, as actually run: compiling and simulating this testbench, the isolated PHY-datapath probe reconstructs the driven FLIT bit-for-bit, and the full `tb_system_top_smoke.v` passes cleanly end to end – **PASS**, all checks (`crc_err`/`fec_err`/ `phy_tx_fifo_overflow` all clear, `sof`/`eof` both asserted with the reconstructed FLIT).

5.3 Protocol–PHY layer

Testbench / method	What it checks	Result
`tb_end_to_end_datapath`	Full symbol-level round trip: striper → per-lane Gray/PAM4 → (ideal channel) → per-lane demap/decode → lane swap → SOF strip → destriper, bit-exact comparison against the original 256-bit FLIT	PASS
`gray.sby`, `pam4.sby`, `verify_reset.sby`	SymbiYosys formal proofs: Gray encoder/decoder are exact inverses across all 4 input combinations; PAM4 map/demap round-trips; reset-sequencing FSM properties	(formal, not re-run in this pass – see Section **??**)
cocotb (`test_pcie_link_top.py`)	Reset holds config/MSI-X quiet; link-up makes config space live; BAR sizing and writes; MSI-X masked/unmasked behavior	**4/4 PASS**, 0 errors, 0 failures

5.4 Link layer

Individually testbenched: tag allocation/recycling/timeout (`tb_tag_manager.v`), credit accounting (`tb_credit_manager.v`), TLP header generation against known-good byte sequences (`tb_tlp_packetizer.v`, `tb_tlp_packetizer_mps.v`), out-of-order completion reassembly (`tb_axi_pcie_reorder_buffer.v`), 4 KB boundary splitting (`tb_ar_request_splitter.v`), BAR decode / UR-on-miss (`tb_address_translation.v`), replay-buffer NAK retransmit (`tb_replay_buffer.v`), and a loopback integration (`tb_link_layer_loopback.v`, `tb_top_pipeline.v`). These have not yet been chained into `system_top.v` (see Section **??**).

6 Conclusion

Each of the three layers has an independently verified RTL implementation with passing unit-level testbenches, and `Final/system_top.v` establishes the first system-level top chaining all three layers – including the PHY datapath and the PCIe control plane – with a connectivity smoke test demonstrating a beat propagating from the AXI4-Stream ingest ports through arbitration, CDC, session gating, FLIT framing, and the full Gray/PAM4 lane encode-decode chain, and back out as a reconstructed FLIT with clean CRC/FEC/overflow status. The open items above – principally the missing TLP packetizer stage, the unresolved session-arm CDC, and the entirety of Stage 2 – are the concrete remaining work before this design can be considered a complete, tape-out-ready Stage 1/Stage 2 submission.